# Frontend Practice Tasks - Complete Challenge List

## 📦 Section 1: Data Mapping & Display (5 Tasks)

### Task 1: User Directory

**Goal:** Display a list of users with profile information

- [ ] Fetch users from JSONPlaceholder API (`/users`)
- [ ] Create a UserCard component showing: name, email, phone, company name
- [ ] Add a loading spinner while fetching
- [ ] Handle API errors with a friendly message
- [ ] Style with grid layout (responsive: 1 col mobile, 3 cols desktop)

### Task 2: Blog by Author

**Goal:** Group blog posts by author

- [ ] Fetch posts from JSONPlaceholder (`/posts` and `/users`)
- [ ] Group posts by `userId` using JavaScript's `reduce()` or `Map`
- [ ] Display author name as section header
- [ ] Show each author's posts in a collapsible accordion
- [ ] Add post count badge next to author name

### Task 3: Nested Data Extraction

**Goal:** Safely handle deeply nested API responses

- [ ] Create mock data with structure: `user.profile.settings.theme.colors.primary`
- [ ] Extract nested values using optional chaining (`?.`)
- [ ] Use destructuring to get multiple nested values
- [ ] Create a utility function `getSafeValue(obj, path, defaultValue)`
- [ ] Display a profile card showing extracted nested data

### Task 4: Polymorphic Card Renderer

**Goal:** Render different components based on content type

- [ ] Create data array with mixed types: `{type: 'video', ...}`, `{type: 'article', ...}`
- [ ] Build `VideoCard`, `ArticleCard`, and `ImageCard` components
- [ ] Create a `CardRenderer` that conditionally renders based on `type`
- [ ] Add TypeScript discriminated unions for type safety
- [ ] Display all cards in a masonry-style grid

**Task 5: Sortable Product Table**

**Goal:** Build an interactive data table

- [ ] Fetch products from FakeStore API
- [ ] Display in table: image, title, price, category, rating
- [ ] Implement sort by clicking column headers (price, rating, title)
- [ ] Add ascending/descending toggle with arrow icons
- [ ] Highlight sorted column with different background color

---

## 📋 Section 2: Forms & Validation (5 Tasks)

**Task 6: Multi-Step Registration Form**

**Goal:** Create a wizard-style form

- [ ] Step 1: Personal info (name, email, phone)
- [ ] Step 2: Address (street, city, country, zip)
- [ ] Step 3: Preferences (newsletter checkbox, interests multi-select)
- [ ] Add "Next", "Back", and "Submit" buttons with proper navigation
- [ ] Show progress indicator (e.g., "Step 2 of 3")
- [ ] Store all data in state and display summary on final step

**Task 7: Validated Login Form**

**Goal:** Form validation with React Hook Form + Zod

- [ ] Install `react-hook-form` and `zod`
- [ ] Create schema: email (valid format), password (min 8 chars)
- [ ] Show inline error messages below each field
- [ ] Disable submit button until form is valid
- [ ] Show success message on submit with form data

**Task 8: Dynamic Field Manager**

**Goal:** Add/remove form fields dynamically

- [ ] Create "Add Education" button
- [ ] Each education entry has: school name, degree, year
- [ ] Add "Remove" button for each entry
- [ ] Minimum 1 entry required, maximum 5 entries

- [ ] Display all entries in a summary list below form

**Task 9: Image Upload with Preview**

**Goal:** Handle file uploads

- [ ] Create file input with "Choose Image" button
- [ ] Show image preview immediately after selection
- [ ] Display file name and size
- [ ] Add "Remove Image" button to clear selection
- [ ] Validate: only images (jpg, png), max 5MB
- [ ] Show validation errors if file is invalid

**Task 10: Cascading Dropdown Form**

**Goal:** Location selector with dependent dropdowns

- [ ] Country dropdown (use mock data: USA, Canada, UK, India)
- [ ] State/Province dropdown updates based on country
- [ ] City dropdown updates based on state
- [ ] Store data in nested object: `{country, state, city}`
- [ ] Display selected location below form

---

## 🎨 Section 3: UI Components (5 Tasks)

**Task 11: Button Component Library**

**Goal:** Build a reusable button system

- [ ] Create `Button` component with variants: primary, secondary, danger, outline
- [ ] Add sizes: small, medium, large
- [ ] Support loading state with spinner
- [ ] Support disabled state
- [ ] Create demo page showing all combinations

**Task 12: Modal Component**

**Goal:** Reusable modal with animations

- [ ] Build `Modal` component with open/close state
- [ ] Add backdrop click to close
- [ ] Add ESC key to close

- [ ] Animate entrance/exit with CSS transitions
- [ ] Include header, body, footer slots
- [ ] Create demo with "Open Modal" button

**Task 13: Star Rating Component**

**Goal:** Interactive rating widget

- [ ] Display 5 stars (filled/unfilled)
- [ ] Click a star to select rating
- [ ] Hover to preview rating
- [ ] Show rating value (e.g., "3 out of 5 stars")
- [ ] Support half-star ratings (bonus)

**Task 14: Product Card Component**

**Goal:** Reusable card with actions

- [ ] Display: image, title, description, price
- [ ] Add "Add to Cart" button
- [ ] Add "Wishlist" heart icon toggle
- [ ] Add "Share" button (just show alert for now)
- [ ] Include badge for "Sale" or "New" items

**Task 15: Tooltip Component**

**Goal:** Hover-triggered info popup

- [ ] Create `Tooltip` wrapper component
- [ ] Show tooltip on hover with 300ms delay
- [ ] Support top, bottom, left, right positions
- [ ] Add arrow pointer
- [ ] Demo with multiple tooltips on a page

---

## 🔄 Section 4: API & Async Operations (5 Tasks)

**Task 16: Debounced Search**

**Goal:** Live search with API calls

- [ ] Create search input for movies (use OMDB or TMDB API)
- [ ] Debounce input by 500ms using custom hook or lodash

- [ ] Show loading indicator while searching
- [ ] Display results in a dropdown/list below input
- [ ] Handle "No results found" state

**Task 17: Paginated API Display**

**Goal:** Navigate through pages of data

- [ ] Fetch posts from JSONPlaceholder with `_page` and `_limit` params
- [ ] Display 10 posts per page
- [ ] Add "Previous" and "Next" buttons
- [ ] Show current page number and total pages
- [ ] Disable buttons appropriately (e.g., "Previous" on page 1)

**Task 18: Infinite Scroll Gallery**

**Goal:** Load more data on scroll

- [ ] Fetch photos from Unsplash or Picsum
- [ ] Display in grid layout
- [ ] Use `IntersectionObserver` to detect when user reaches bottom
- [ ] Load next batch automatically
- [ ] Show loading spinner at bottom while fetching

**Task 19: Error Handling & Retry**

**Goal:** Robust API error management

- [ ] Fetch data from an API (can use a mock failing endpoint)
- [ ] Show error message if fetch fails
- [ ] Add "Retry" button
- [ ] Implement exponential backoff (retry after 1s, 2s, 4s)
- [ ] Show success message when data loads

**Task 20: Auto-Refresh Dashboard**

**Goal:** Polling for fresh data

- [ ] Fetch cryptocurrency prices or weather data
- [ ] Auto-refresh every 30 seconds using `setInterval`
- [ ] Show "Last updated: X seconds ago" timestamp
- [ ] Add "Refresh Now" button for manual refresh
- [ ] Clean up interval on component unmount

---

## 🌍 Section 5: Real-World Features (5 Tasks)

### Task 21: Shopping Cart System

**Goal:** Full cart with add/remove/quantity

- [ ] Create product list (5-10 items)
- [ ] Add "Add to Cart" button on each product
- [ ] Show cart icon with item count in header
- [ ] Cart page shows: product, quantity, price, subtotal
- [ ] Include +/- buttons to adjust quantity
- [ ] Display total price
- [ ] "Remove from Cart" button
- [ ] Store cart in Context API or localStorage

### Task 22: Dark/Light Theme Toggle

**Goal:** Global theme management

- [ ] Create ThemeContext with light/dark modes
- [ ] Add toggle switch in header
- [ ] Store theme preference in localStorage
- [ ] Apply theme using CSS variables or Tailwind dark mode
- [ ] Animate theme transition smoothly

### Task 23: Notification System

**Goal:** Toast/snackbar notifications

- [ ] Create NotificationContext
- [ ] Build `Toast` component (success, error, warning, info)
- [ ] Add `showNotification(message, type)` function
- [ ] Auto-dismiss after 3 seconds
- [ ] Support multiple notifications stacked
- [ ] Include close button

### Task 24: Responsive Dashboard Layout

**Goal:** Professional admin panel structure

- [ ] Create sidebar with navigation links
- [ ] Collapsible sidebar on mobile (hamburger menu)

- [ ] Top header with user profile and logout
- [ ] Main content area with grid of stat cards
- [ ] Display charts (can use Recharts library)
- [ ] Fully responsive: mobile, tablet, desktop

**Task 25: YouTube-Style Video Grid**

**Goal:** Clone a real-world UI

- [ ] Fetch video data (can use YouTube API or mock data)
- [ ] Display thumbnail, title, channel name, views, upload date
- [ ] Grid layout: 1 col mobile, 2 cols tablet, 4 cols desktop
- [ ] Add hover effect: show play button overlay
- [ ] Include channel avatar image
- [ ] Add duration badge on thumbnail

---

## 🎯 How to Approach These Tasks

1. **Do 3-4 tasks per week -** quality over speed
2. **Start simple, then enhance -** get it working first, then improve
3. **Use TypeScript -** define types for props and data
4. **Keep a GitHub repo -** commit after completing each task
5. **Write a README -** explain what you learned in each task
6. **Use real APIs** when possible:
    - JSONPlaceholder (posts, users)
    - FakeStore API (products)
    - TMDB/OMDB (movies)
    - Unsplash/Picsum (images)
    - REST Countries API (locations)

---

## 📚 Bonus Challenges (When You Finish)

- Add unit tests using Vitest or Jest
- Deploy each task to Vercel/Netlify
- Combine multiple tasks into one large project

- Refactor to use React Query or SWR for data fetching
- Add animations with Framer Motion
- Build mobile-first and ensure accessibility (ARIA labels)

---

**তুমি পারবে! 💪 একটা করে task শেষ করো, GitHub এ push করো, এবং শিখতে থাকো। Good luck!**